

NumPy for Machine Learning

Практический конспект • Shapes • Vectorization • Broadcasting • Linear Algebra

Главная структура ML-данных

$$X.shape = (n_{\text{samples}}, n_{\text{features}})$$

Обычно:

$$y.shape = (n_{\text{samples}},), \quad w.shape = (n_{\text{features}},)$$

Содержание

1	ndarray и Shape	1
2	Критическая разница Shapes	1
3	Indexing и Boolean Indexing	1
4	Reshape и Transpose	2
5	Elementwise Operations	2
6	Aggregations и Axis	2
7	Broadcasting	3
8	Стандартизация	3
9	Dot Product и Matrix Multiplication	3
10	Главная ML-операция	4
11	Vectorization	4
12	Random, Seed и Shuffle	4
13	Copy и View	5
14	Финальная шпаргалка	5
15	Что должно быть в голове	6

1. ndarray и Shape

```
import numpy as np

a = np.array([1, 2, 3])

a.shape
a.ndim
a.size
a.dtype
```

Важно: shape, ndim, size, dtype — атрибуты.

a.shape ✓
a.shape() ×

2. Критическая разница Shapes

$$(n,) \neq (n, 1) \neq (1, n)$$

```
a = np.array([1, 2, 3])

a.shape           # (3,)
a.reshape(-1, 1).shape # (3, 1)
a.reshape(1, -1).shape # (1, 3)
```

ML: один feature для n samples часто нужно передавать как

$$(n, 1)$$

например:

```
age = age.reshape(-1, 1)
```

3. Indexing и Boolean Indexing

```
x[1, 2]      # element
x[1, :]      # row
```

```
X[:, 1]      # column
X[1:4, 1:3]  # slice
```

Фильтрация:

```
X[X[:, 0] >= 18]

mask = (age >= 18) & (income >= 80000)
X[mask]
```

Для NumPy-массивов:

$\& = AND, \quad | = OR, \quad \sim = NOT$

Каждое сравнение заключается в скобки:

```
(x >= 10) & (x <= 20)
```

4. Reshape и Transpose

```
X.reshape(3, 4)
X.reshape(-1, 4)

X.T
```

Если:

$$X.shape = (100, 20)$$

то:

$$X^T.shape = (20, 100)$$

Для 1D-вектора transpose ничего не меняет:

```
a.shape      # (3,)
a.T.shape     # (3,)
```

Чтобы получить настоящий column vector:

```
a.reshape(-1, 1)
```

5. Elementwise Operations

```
X + 5
X * 2
X ** 2

A + B
A - B
A * B
A / B

np.sqrt(X)
np.exp(X)
np.log(X)
np.abs(X)
```

$*$ = elementwise

$@$ = matrix multiplication

6. Aggregations и Axis

```
X.sum()
X.mean()
X.std()

X.min()
X.max()

X.argmin()
X.argmax()
```

`max()` возвращает значение, `argmax()` — индекс.

Правильное понимание axis:

`axis=N` $\Rightarrow$ сворачиваем ось N

Для:

$$X.shape = (4, 3)$$

получаем:

$$X.mean(axis = 0) \rightarrow (3,)$$

одно значение на feature,

$$X.mean(axis = 1) \rightarrow (4,)$$

одно значение на sample.

7. Broadcasting

Размерности сравниваются **справа налево**.

Они совместимы, если:

$$a = b \quad \text{или} \quad a = 1 \quad \text{или} \quad b = 1$$

Пример:

$$(4, 3) + (3,)$$

мысленно превращается в:

$$(4, 3) + (1, 3) \rightarrow (4, 3)$$

Ещё примеры:

$$(100, 20) + (20,) \rightarrow (100, 20)$$

$$(100, 20) + (100,) \rightarrow ERROR$$

$$(4, 1) + (1, 3) \rightarrow (4, 3)$$

8. Стандартизация

$$x' = \frac{x - \mu}{\sigma}$$

```
mean = X.mean(axis=0)
std = X.std(axis=0)

X_scaled = (X - mean) / std
```

Broadcasting применяет отдельные μ_j и σ_j к каждому feature.

После этого:

$$mean(X_{\text{scaled}}) \approx 0, \quad std(X_{\text{scaled}}) \approx 1$$

Стандартизированное значение показывает, на сколько стандартных отклонений объект находится выше или ниже среднего.

9. Dot Product и Matrix Multiplication

Для векторов:

$$x \cdot w = \sum_j x_j w_j$$

```
x @ w
np.dot(x, w)
(x * w).sum()
```

Для матриц:

$$(m, n) @ (n, k) \rightarrow (m, k)$$

Правило:

внутренние размеры должны совпасть, внешние остаются.

$$(m, n) @ (n, k) = (m, k)$$

10. Главная ML-операция

$$z = Xw + b$$

```
z = X @ w + b
```

Если:

$$X.shape = (100, 20), \quad w.shape = (20,)$$

то:

$$(100, 20) @ (20,) \rightarrow (100,)$$

То есть получаем одно значение для каждого sample.

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

```
p = 1 / (1 + np.exp(-z))
```

Prediction:

```
y_pred = (p >= 0.5).astype(int)
```

Accuracy:

```
accuracy = (y_pred == y).mean()
```

11. Vectorization

Вместо:

```
for i in range(len(X)):
    ...
```

по возможности работаем сразу со всем массивом:

```
y_pred = X @ w + b

mse = ((y_pred - y) ** 2).mean()
```

Vectorization не означает “никогда не использовать for”.

Цикл по эпохам нормален. Лишние циклы по samples/features обычно заменяются операциями NumPy.

12. Random, Seed и Shuffle

Современный генератор:

```
rng = np.random.default_rng(42)
```

```
rng.random(5)
```

```
rng.integers(0, 10, size=5)
```

```
rng.normal(
```

```
loc=0,
scale=1,
size=(100, 5)
)
```

Seed нужен для воспроизводимости.

Нельзя независимо перемешивать X и y :

```
rng.shuffle(X)
rng.shuffle(y)
```

Так можно разрушить связь:

$$X[i] \leftrightarrow y[i]$$

Правильно:

```
indices = rng.permutation(len(X))

X_shuffled = X[indices]
y_shuffled = y[indices]
```

13. Copy и View

```
b = a
```

— тот же объект.

Slice часто является view:

```
b = a[1:4]
b[0] = 999
```

Изменение b может изменить a .

Независимая копия:

```
b = a[1:4].copy()
```

Также:

```
X.ravel() # view
X.flatten() # copy
```

Если собираешься менять производный массив и оригинал должен гарантированно сохраниться:

```
new_X = X.copy()
```

14. Финальная шпаргалка

```
# Dataset
X.shape
X[:, 0]
X[mask]

# Shape
x.reshape(-1, 1)
X.T

# Statistics
X.mean(axis=0)
X.std(axis=0)
X.argmax(axis=1)

# Scaling
X_scaled = (
    X - X.mean(axis=0)
) / X.std(axis=0)

# Linear model
z = X_scaled @ w + b

# Sigmoid
p = 1 / (1 + np.exp(-z))

# Prediction
y_pred = (p >= 0.5).astype(int)
```

```
# Accuracy
accuracy = (y_pred == y).mean()

# Random
rng = np.random.default_rng(42)

# Joint shuffle
idx = rng.permutation(len(X))
X_new = X[idx]
y_new = y[idx]

# Independent copy
X_copy = X.copy()
```

15. Что должно быть в голове

$$X : (n_{\text{samples}}, n_{\text{features}})$$

$$(n,) \neq (n, 1) \neq (1, n)$$

$$\text{axis} = N \Rightarrow \text{collapse axis } N$$

Broadcasting: compare shapes right-to-left

* = elementwise @ = matrix multiplication

$$(m, n) @ (n, k) \rightarrow (m, k)$$

Типичный NumPy-фрагмент ML pipeline:

$$X \rightarrow X_{\text{scaled}} \rightarrow Xw + b \rightarrow \sigma(z) \rightarrow \hat{y} \rightarrow \text{metric}$$